Lesson 4: Building the Retrieval Pipeline — RAG Version 1 (Day 31)

> ***Goal:*** *In Lesson 2, we learned how to talk to LLMs. In Lesson 3, we chopped our documents into chunks. Today, we connect all the puzzle pieces to build* ***RAG (Retrieval-Augmented Generation)****. We will store chunks in a persistent* ***Chroma vector database****, search them by meaning, and generate answers backed by facts.*

□ What is RAG? (The Open-Book Exam Analogy)

Imagine you have to take an exam:

- **Closed-Book Exam:** You memorize a 1,000-page book for weeks. During the test, you try to remember from memory. You might misremember numbers or make things up (hallucination).
- **Open-Book Exam (RAG):** You don't memorize the book! You bring the book with you. When a question is asked:
    1. **Retrieve:** You flip through the index and find the **exact 2 pages** that discuss that topic.
    2. **Augment:** You read those 2 pages and hold them in your mind alongside the question.
    3. **Generate:** You write down a clear, accurate answer based *only* on those pages.

```
User Question: "How many sick days do I get?"
       |
       ▼
[ Step 1: RETRIEVE ]  ⟶ ChromaDB searches 15 chunks using embeddings
                         Finds the 3 closest chunks about sick leave
       |
       ▼
[ Step 2: AUGMENT ]   ⟶ Python glues the 3 chunks into the prompt:
                         "Context: An employee gets 10 sick days...
                          Question: How many sick days do I get?"
       |
       ▼
[ Step 3: GENERATE ]  ⟶ LLM reads the context and writes:
                         "You get 10 sick days in one year."
```

□ Vocabulary: Terms You Must Know

Word	What it Means	Everyday Real-World Analogy
RAG	**R**etrieval-**A**ugmented **G**eneration: search first, add context, then answer.	An open-book exam.
Retrieve	Finding and fetching the closest chunks from the vector database.	Finding the right page in a library book.
Augment	Injecting the retrieved chunks into the prompt text before sending it to the AI.	Handing the book page to the person writing the answer.
Generate	The LLM reading the injected text and producing the final response.	Writing the final answer on your test sheet.
ChromaDB	An open-source, fast vector database designed to store embeddings and search by cosine similarity.	A specialized library filing cabinet that indexes concepts instead of alphabetized names.
PersistentClient	A Chroma setting that saves the database to a **folder on your hard drive** (`company_db`) so it isn't lost when Python shuts down.	Saving a Word document to your hard drive instead of leaving it unsaved in RAM.
Cosine Distance	A number measuring how far apart two vectors are. `0.0` = identical meaning; `1.0` = completely different.	The physical distance between two cities on a map.

□ Step-by-Step Architecture

```
[ Step 1: Load & Chunk ]    company_docs/*.txt ⟶ 15 clean chunks (Day 30 code)
                                        |
                                        ▼
[ Step 2: Embed & Store ]   Chroma PersistentClient ("company_db")
                            Embed each chunk ONCE with NVIDIA embedder. Save to disk.
                                        |
                                        ▼
[ Step 3: Retrieve ]        User asks question ⟶ Chroma queries top 3 by cosine distance
                                        |
                                        ▼
[ Step 4: Prompt & Ask ]    Build prompt: System Rule + Context Chunks + Question
                                        |
                                        ▼
[ Step 5: Final Answer ]    LLM outputs grounded response!
```

Step 1: Setup & API Clients

We need the `openai` client for calling OpenRouter and `chromadb` for our vector store:

```
import os
from pathlib import Path
from dotenv import load_dotenv
from openai import OpenAI
import chromadb

# Load environment secrets
load_dotenv()
client = OpenAI(
    base_url="https://openrouter.ai/api/v1",
    api_key=os.getenv("OPEN_ROUTER_KEY")
)

MODEL = "nvidia/nemotron-3.5-lightning:free"
EMB_MODEL = "nvidia/nemotron-3-embed-1b:free"

def ask(messages, temperature=0):
    try:
        r = client.chat.completions.create(model=MODEL, messages=messages, temperature=temperature)
        return (r.choices[0].message.content or "").strip()
    except Exception as e:
        return f"ERROR: {e}"

def embed(texts):
    # Sends a list of strings, receives back a list of 2048-dimensional vectors
    r = client.embeddings.create(model=EMB_MODEL, input=texts)
    return [item.embedding for item in r.data]
```

Step 2: Load and Chunk Documents (Day 30 Pipeline)

```
def load_documents(folder):
    docs = []
    for path in sorted(Path(folder).glob("*.txt")):
        docs.append({"source": path.name, "text": path.read_text(encoding="utf-8")})
    return docs

def chunk_by_paragraph(text):
    parts = [p.strip() for p in text.split("\n\n")]
    return [p for p in parts if len(p) > 30]

def chunk_documents(docs):
    chunks = []
    for doc in docs:
        for i, text in enumerate(chunk_by_paragraph(doc["text"])):
            chunks.append({
                "id": f"{doc['source']}-{i}",
                "source": doc["source"],
                "text": text
            })
    return chunks

chunks = chunk_documents(load_documents("company_docs"))
print(f"Prepared {len(chunks)} chunks.")
```

Step 3: Persistent Storage in ChromaDB

On Day 29, Chroma lived only in memory. If you closed Python, your database vanished.

Today we use `chromadb.PersistentClient(path="company_db")`.

⚠ The Cost-Saving Trick: `if store.count() == 0`

Free accounts get ~50 API requests per day. If you run your notebook 10 times and re-embed all 15 chunks every time, you will burn all your quota in 3 minutes!

We wrap the upload in a check:

```
# 1. Connect to local folder "company_db"
db = chromadb.PersistentClient(path="company_db")

# 2. Create or open the collection with cosine distance
store = db.get_or_create_collection(
    name="company_docs",
    metadata={"hnsw:space": "cosine"}   # Tells Chroma to use cosine distance
)

# 3. Only embed if the database is currently empty!
if store.count() == 0:
    print("Database is empty. Embedding chunks for the first time...")

    ids = [c["id"] for c in chunks]
    documents = [c["text"] for c in chunks]
    metadatas = [{"source": c["source"]} for c in chunks]

    store.add(
        ids=ids,
        documents=documents,
        metadatas=metadatas,
        embeddings=embed(documents)   # Generates vectors in one API call
    )
    print("Embedded and saved!")
else:
    print(f"Database already contains {store.count()} chunks. Skipping embedding!")
```

□ Why `hnsw:space: "cosine"`?

By default, Chroma uses Euclidean distance (\(L_2\)). For text embeddings, **Cosine Distance** is the industry standard because it compares the *direction/meaning* of vectors regardless of their length.

Step 4: Building the `retrieve()` Function

When a user asks a question:

1. We embed the question string into a vector.
2. We ask Chroma for the `n=3` closest chunks.
3. We unpack the results into a clean list of dictionaries:

```
def retrieve(question, n=3):
    # 1. Convert question to vector and search Chroma
    r = store.query(
        query_embeddings=embed([question]),
        n_results=n
    )

    # 2. Chroma returns lists of lists; extract the first result set
    texts = r["documents"][0]
    sources = r["metadatas"][0]
    distances = r["distances"][0]

    found = []
    for i in range(len(texts)):
        found.append({
            "text": texts[i],
            "source": sources[i]["source"],
            "distance": distances[i]
        })
    return found

# Test retrieval:
results = retrieve("How many sick days do I get?")
for r in results:
    print(f"Dist: {r['distance']:.2f} | [{r['source']}] | {r['text'][:60]}...")
```

Output:

```
Dist: 0.18 | [leave_policy.txt] | Sick leave is separate from paid leave. An employee gets 10...
Dist: 0.45 | [leave_policy.txt] | Every employee gets 20 days of paid leave in one year...
Dist: 0.62 | [working_hours.txt] | If you arrive after 9:30 am, you are marked late...
```

Notice that the top result has distance `0.18` (very close!) and directly talks about sick leave.

Step 5: Building the `rag()` Function (The Full Pipeline)

Now we join Retrieve, Augment, and Generate:

```
RAG_SYSTEM = """You are the HR support bot of a company.
Answer the employee's question using ONLY the provided context.
Be friendly. Reply in at most 3 short sentences."""

def rag(question):
    # 1. RETRIEVE: Get top 3 chunks
    found = retrieve(question, n=3)

    # 2. AUGMENT: Stitch chunks together into a context block
    context = "\n\n".join(f["text"] for f in found)

    # Build prompt messages
    messages = [
        {"role": "system", "content": RAG_SYSTEM},
        {"role": "user", "content": f"Context:\n{context}\n\nQuestion: {question}"}
    ]

    # 3. GENERATE: LLM answers based on the context
    return ask(messages)

print(rag("How many sick days do I get?"))
```

Bot Answer:

> *"You get 10 sick days in one year, which are separate from your paid leave. If you take more than 2 sick days in a row, you must provide a doctor's note."*

Step 6: Troubleshooting & The "Dog" Problem

What happens if an employee asks:
"Can I bring my dog to the office?"

Let's inspect what happens under the hood with a debugging helper:

```
def debug(question):
    print("=" * 60)
    print("QUESTION:", question)
    found = retrieve(question)
    for f in found:
        print(f"  -> Found (dist: {f['distance']:.2f}) [{f['source']}]: {f['text'][:50]}...")
    print("BOT ANSWER:", rag(question))

debug("Can I bring my dog to the office?")
```

Output:

```
QUESTION: Can I bring my dog to the office?
  -> Found (dist: 0.89) [working_hours.txt]: The office is open from 9 am to 5 pm...
  -> Found (dist: 0.91) [laptop_rules.txt]: Every new employee gets one company laptop...
  -> Found (dist: 0.94) [expense_policy.txt]: The company pays back travel costs...
BOT ANSWER: The office is open from 9 am to 5 pm Monday to Friday. Please follow office rules.
```

□ The Big Gotcha of RAG v1:

- Chroma **ALWAYS returns the 3 closest chunks**, even if all of them are 100% irrelevant!
- When you ask about dogs, Chroma returns chunks about laptops and office hours because those were the closest things it had.
- The LLM, trying to be helpful, might hallucinate or give a weird answer.
- **How to fix this?** In Day 32, we add a **Distance Cutoff Threshold** to reject questions that are too far away!

□ The RAG Troubleshooting Matrix

When your RAG bot gives a bad or wrong answer, **do not immediately blame the LLM!** Diagnose which half of the pipeline failed:

Symptom	Which Stage Failed?	How to Fix It
The right document was never in the 3 retrieved chunks	**Retrieve**	Check chunking strategy; increase `n=5`; check for typos in documents.
The right chunk WAS retrieved, but the bot ignored it	**Generate**	Tighten system prompt; lower temperature to 0; make context instructions clearer.
The bot answers questions about topics not in your files	**Guardrail / Filter**	Add a distance cutoff (Day 32); enforce "Say I don't know".
`store.count()` is 0	**Database Storage**	Re-run embedding cell; ensure file path exists.
`429 RateLimitError`	**API Quota**	You exceeded daily free requests; wait for quota reset or avoid re-embedding.

□ Key Takeaways

1. **RAG = Search + Prompting:** Vector DB finds facts, Python injects them into the prompt, LLM synthesizes the answer.
2. **PersistentClient is mandatory:** Saves embeddings on disk in `company_db` so you never waste API calls re-embedding the same files.
3. **Embeddings are decoupled from Generation:** Embedding model (`nemotron-embed`) searches; chat model (`nemotron-lightning`) writes.
4. **Distance matters:** Small distance (\(< 0.4\)) means a strong match; high distance (\(> 0.8\)) means Chroma found nothing relevant.